Guía del notebook `3_2_entrenar_capa_conv`: reto, entrenar un kernel convolucional

Puntos: reto, abierto · **Tiempo:** 45–90 min · **Módulo:** puente a redes profundas (clase 7) · **Archivo:** `notebooks/3_2_entrenar_capa_conv.ipynb`
Requisito: `3_1` (correlación cruzada) y `1_07` (descenso por gradiente).

¿Qué vas a aprender?

- Que los números de un kernel se pueden **aprender** con descenso por gradiente, igual que los pesos de una neurona.
- A calcular el gradiente de una capa convolucional.
- Trucos prácticos que hacen que un entrenamiento real funcione: **normalizar**, **centrar**, **vectorizar** y usar **momentum**.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/3_2_entrenar_capa_conv.ipynb` → kernel **nni**.
   Usa `files/Lenna.png`, `files/sobel_horizontal.png` y el paquete `nni`.

El reto

Tenemos:

- la **entrada**: el canal rojo de la foto (256 × 256);
- el **objetivo**: la misma foto procesada con un detector de **bordes horizontales** (filtro de Sobel), también 256 × 256.

Hay que encontrar los **25 números de un kernel de 5 × 5** tales que ReLU(correlación(kernel, entrada)) ≈ objetivo.

Es el problema de la Clase 4, con dos cambios:

- los "pesos" son los 25 números del kernel;
- la "predicción" es una imagen entera.

La idea: el gradiente de un peso del kernel

Llamemos:

- G = correlación(K, I);
- P = ReLU(G), la predicción;
- T, el objetivo;
- ECM = promedio de $(P - T)^2$.

Un número del kernel, K[u, v], participa en **todos** los píxeles: en cada uno se multiplica por I[i + u, j + v]. Por la regla de la cadena:

∂ECM/∂K[u, v] = (2/N) · Σ sobre todos los píxeles (i, j) de: (P − T)[i, j] · [G[i, j] > 0] · I[i + u, j + v]

- (P − T) es el error en ese píxel.
- [G > 0] vale 1 si la ReLU dejó pasar el valor y 0 si no: la derivada de la ReLU.
- I[i + u, j + v] es el píxel que se multiplicó por ese peso.

Luego se aplica el descenso por gradiente: **K ← K − η·gradiente**.

Paso a paso

Celdas 1–3: imágenes

✅ **Comprobación:** ambas imágenes miden **(256, 256)**, son `uint8`, y el objetivo va de 0 a 255.

Celda: kernel aleatorio y pase frontal

El kernel es aleatorio y la celda no fija semilla, así que el resultado cambia en cada ejecución. Muy a menudo la predicción sale **toda en negro** (mínimo 0.0 y máximo 0.0): como todos los píxeles de la foto son positivos, si los 25 números del kernel suman un valor negativo, casi todos los G son negativos y la ReLU los deja en 0.

Así **ningún peso recibe gradiente** (es una "ReLU muerta"), y el descenso no puede empezar. Es la primera trampa del reto.

✅ **Comprobación:** cuando la predicción sale toda negra, el `Error cuadrático medio` es ≈ **1185.46**, que es exactamente el error de predecir todo en 0. Si te sale otro número, tu kernel aleatorio dio valores positivos: es normal.

Celdas con las funciones sugeridas

El notebook propone calcular el gradiente **píxel por píxel** con tres funciones auxiliares. Eso es correcto, pero son 252 × 252 píxeles × 25 pesos ≈ 1.6 millones de operaciones de Python **por época**: tardaría minutos por época.

Estrategia que funciona (probada)

1. **Normaliza:** divide la imagen y el objetivo entre 255, para que queden entre 0 y 1. Así los gradientes no son gigantes.
2. **Centra la entrada:** réstale su promedio. Así G ya no es "todo negativo" al inicio y la ReLU no muere. Como el kernel buscado suma 0, centrar no cambia la solución.
3. **Vectoriza:** en lugar de recorrer píxel por píxel, prepara las 25 versiones **desplazadas** de la imagen, una por cada (u, v). Por ejemplo, `I[k+u : m-k+u, k+v : n-k+v]`.
   - La correlación completa es la suma de cada versión desplazada por su peso.
   - El gradiente de cada peso es el promedio de (error · máscara · su versión desplazada) por 2.
   - Todo con operaciones de NumPy sobre matrices completas: cada época tarda milisegundos.
4. **Usa momentum** (notebook 22): V ← β·V + gradiente y K ← K − η·V, con β = 0.9. La imagen tiene píxeles vecinos muy parecidos, lo que hace el valle del error muy alargado: justo donde el momentum ayuda.
5. **Hiperparámetros** que funcionaron:
   - kernel inicial `np.random.normal(size=(5, 5))` con semilla 42;

- tasa η = 2.0;
- β = 0.9;
- **2000 épocas**, unos 5 segundos.

✅ **Resultado de referencia:**

- **ECM:** de ≈ 31 600 a ≈ **29**, en unidades de píxel².
- **Kernel aprendido:** prácticamente el **filtro de Sobel** (bordes horizontales), en el centro del 5 × 5:

```
[[ 0.05 -0.01 -0.09 -0.02  0.07]
 [-0.03 -0.86 -1.77 -0.89 -0.11]
 [-0.05 -0.03 -0.01  0.04  0.06]
 [ 0.05  0.93  1.79  0.84  0.06]
 [-0.05 -0.01  0.08  0.03 -0.08]]
```

  Compáralo con Sobel: fila de arriba −1, −2, −1; centro 0; fila de abajo +1, +2, +1.
- **Imagen:** la predicción es casi idéntica al objetivo.

Para pensar

- **¿Por qué la red "descubrió" el filtro de Sobel?** Porque el objetivo se hizo con él. Una CNN real aprende sus filtros del mismo modo, pero el objetivo es una tarea (reconocer gatos, por ejemplo) y no una imagen dada.
- **¿Por qué funciona mejor con momentum?** Sin él, con tasas grandes el entrenamiento rebota y "mata" la ReLU; con tasas pequeñas avanza muy lento.

Errores frecuentes

Síntoma	Causa	Solución
La predicción siempre es negra y el ECM no cambia	ReLU muerta: todos los G son negativos	Centra la entrada; inicializa con valores pequeños
Cada época tarda minutos	Cálculo píxel por píxel con `for`	Vectoriza con las 25 versiones desplazadas
El ECM sale `nan` o explota	Tasa demasiado grande	Baja η, o normaliza entre 0 y 1 si no lo hiciste
ECM gigantesco al inicio	Trabajas con valores 0–255	Divide imagen y objetivo entre 255
La forma no coincide con el objetivo	La correlación válida mide 252 × 252	Recorta el objetivo igual: `T[k:m-k, k:n-k]`